

C code:

```
int64_t    x28[];    // double array
int32_t    x22[];    // word array
int32_t    x1 = 0;    // loop variable

while(x1 < 10){
    x22[x1] = (int32_t) x28[x1];
    x1++;
}
x1 = -100;
```

Assembly code:

```
                addi x1, x0, 0                // x1 = 0
while_loop:
                blt x1, 10, loop_logic
                beq x0, x0, exit                // exit code if x1>=10
loop_logic:
                slli  x8, x1, 3                // x8 = x1*8
                add   x8, x28, x8              // x8 = x28 + x8 (base + offset)
                ld    x18, 0(x8)               // x18 = x28[x1]
                slli  x2, x1, 2                // x2 = x1*4
                add   x2, x22, x2              // x2 = x22 + x2 (base + offset)
                sw    x18, 0(x2)               // x22[x1] = (int32_t) x28[x1]
                addi  x1, x1, 1                // x1++
                beq    x0, x0, while_loop
exit:
                addi  x1, x0, -100              // x1 = -100
```

Summary:

1. See the logic for ld and sw instructions i.e. how arrays are encoded in assembly.
2. In C, array index runs with an increment of 1 (irrespective of double or word type). However, in assembly index changes to offset which runs with an increment of 8 for double (64 bits) and 4 for word (32 bits). We achieve this by using **slli** instruction.
3. We can use blt, bgt, bge or some other instruction for encoding loops, however placement of loop logic and exit condition will change. In the above code, I have added labels for clarity. Try implementing same loop with bgt or bge and same labels. Order of exit label and loop_logic label will change.